COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	08/01/26	**DATE SUBMITTED:**	08/03/26
NAME:	Gabriel Dwyane D. Arididdon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO 4.1 **Hands-on Activity 4.1 Stacks**

1. PROCEDURE OUTPUT

ILO B.2: Stacks using Linked List

push(T newData)

```
//push
template <typename T>
void push(T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}
```

```
int main() {
    std::cout << "---- PUSHING 'c' 'p' 'e' ONTO THE STACK ----\n";
    //push
    push('c');
    push('p');
    push('e');
```

This function adds a new element to the top of the stack. It first creates a new node, stores the given value in it, and then connects it to the current top of the stack. After that, the new node becomes the new top. Since a stack follows the Last In, First Out (LIFO) rule, the most recently added item will always be the first one to come out.

pop()

```
//pop
template<typename T>
T pop(){
    if(head<T> == nullptr){
        std::cout << "stack overflow." << std::endl;
    }
    //save current top node
    Node<T>* temp = head<T>;
    //store data
    T tempVal = temp->data;

    //move head to next node
    head<T> = head<T>->next;
    //free the removed node
    delete temp;
    //return the remove value
    return tempVal;
}
```

```
std::cout << "\n ---- Popping all the elements ----\n";

while(!isEmpty<char>()){
    std::cout << " Popped: " << pop<char>() << std::endl;
}

std::cout << "\n ---- Testing the Underflow ---- \n";
pop<char>();
```

This function removes the element at the top of the stack and returns its value. It first checks if the stack is empty. If it isn't, it saves the top node, moves the top pointer to the next node, deletes the old top node from memory, and returns the value that was removed. If the stack is empty, it displays an underflow message because there's nothing to remove.

isEmpty()

```
//isEmpty
template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}
```

```
std::cout << "\n ---- Popping all the elements ----\n";

while(!isEmpty<char>()){
    std::cout << " Popped: " << pop<char>() << std::endl;
}

std::cout << "\n ---- Testing the Underflow ---- \n";
pop<char>();
```

This function checks whether the stack has any elements. If the top pointer is nullptr, it means the stack is empty, so it returns true. Otherwise, it returns false. This is useful before trying to remove or access the top element.

Top()

```
//top
template<typename T>
void Top(){
    if(isEmpty<T>()){
        std::cout<<"Stack underflow"<<std::endl;
        return;
    }
    std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
}
```

```
//display the current top
Top<char>(); // Top should be 'e'
displayAll<char>();
```

This function displays the value at the top of the stack without removing it. Before showing the value, it checks if the stack is empty. If there are no elements, it prints an underflow message instead.

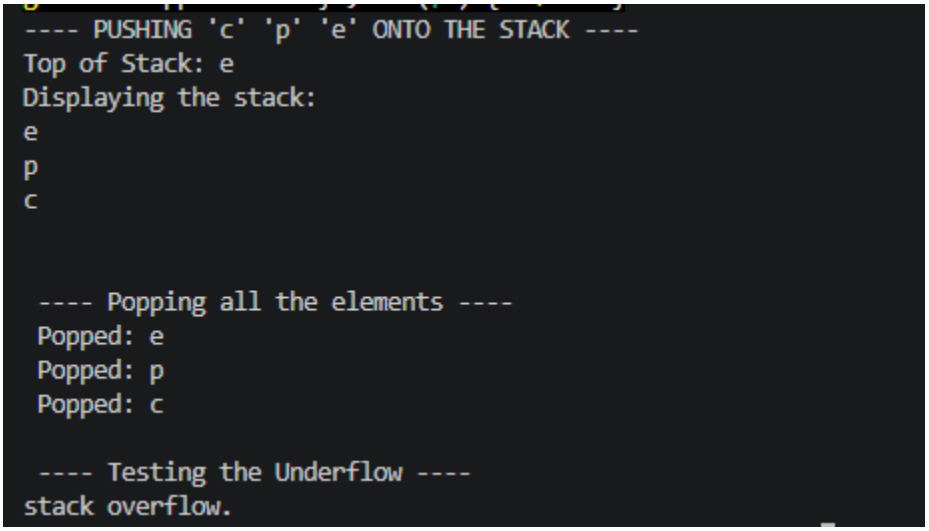
displayAll()

```
//displayAll
template <typename T>
void displayAll(){
    if (head<T> == nullptr){
        std::cout << "stack underflow\n";
        return;
    }
    std::cout << "Displaying the stack: \n";
    Node<T>* temp = head<T>;
    while(temp != nullptr){
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout << "\n";
}
```

```
//display the current top
Top<char>(); // Top should be 'e'
displayAll<char>();
```

This function prints every element in the stack, starting from the top and going down to the bottom. It uses a temporary pointer to go through each node one by one and displays the stored values. If the stack is empty, it lets the user know by printing an underflow message.

Output



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

A. Stack using Arrays

supp.cpp > isFull()

```cpp
#include <iostream>
#include <string>

#define maxCap 100

// Global declarations
char stackArr[maxCap];
int topIndex = -1;


// Prototype functions
void push(char newData);
char pop();
bool isEmpty();
bool isFull();
bool match(char open, char close);
bool balanced(std::string expression);


// Main driver
int main() {

    std::string expression;

    std::cout << "Enter expression: ";
    getline(std::cin, expression);

    if (balanced(expression)) {
        std::cout << "Expression is Balanced." << std::endl;
    }
    else {
        std::cout << "Expression is NOT Balanced." << std::endl;
    }

    return 0;
}


// Check if stack is empty
bool isEmpty() {
    if (topIndex == -1)
        return true;

    return false;
}


// Check if stack is full
bool isFull() {
    if (topIndex == maxCap - 1)
        return true;

    return false;
}


```

```cpp
// Push operation
void push(char newData) {

    if (isFull()) {
        std::cout << "Stack Overflow" << std::endl;
        return;
    }

    stackArr[++topIndex] = newData;
}


// Pop operation
char pop() {

    if (isEmpty()) {
        return '\0';
    }

    return stackArr[topIndex--];
}


// Check matching symbols
bool match(char open, char close) {

    if (open == '(' && close == ')')
        return true;

    if (open == '{' && close == '}')
        return true;

    if (open == '[' && close == ']')
        return true;

    return false;
}
```

```cpp
// Balanced symbol checker
bool balanced(std::string expression) {

    for (char ch : expression) {

        // If opening symbol, push to stack
        if (ch == '(' || ch == '{' || ch == '[') {

            push(ch);

        }

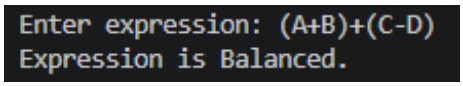
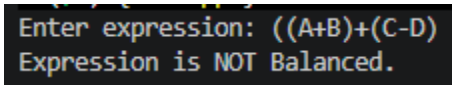
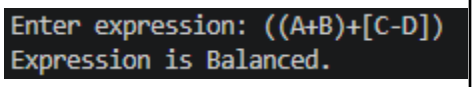
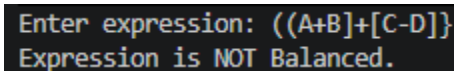
        // If closing symbol, check stack
        else if (ch == ')' || ch == '}' || ch == ']') {

            if (isEmpty())
                return false;

            char temp = pop();

            if (!match(temp, ch))
                return false;
        }
    }


    // If stack is empty, all symbols matched
    return isEmpty();
}
```

Expression	Valid ? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		The program correctly pushes the opening parentheses and removes them when the matching closing parentheses are found. Since all symbols are matched and the stack becomes empty at the end, the expression is valid.
((A+B)+(C-D)	N		There is one extra opening parenthesis that does not have a matching closing parenthesis. After checking the whole expression, the stack still contains a symbol, which means the expression is incomplete.
((A+B)+[C-D])	Y		The stack successfully handles different symbols like parentheses and brackets. Each opening symbol is matched with the correct closing symbol, leaving the stack empty after the checking process.
((A+B]+[C-D]}	N		The closing symbols do not match their corresponding opening symbols. The program detects that the order and type of symbols are incorrect, so the expression is considered invalid.

B. Stack using Linked Lists

suppLL.cpp > main()

```cpp
#include <iostream>
#include "stacklist(modified).h"

int main(){

    std::string expression;

    std::cout << "Enter expression: ";
    getline(std::cin, expression);

    if(balanced<char>(expression))
        std::cout << "Expression is Balanced.\n";
    else
        std::cout << "Expression is NOT Balanced.\n";

    return 0;
}

```

For the header file, I used the stackList.h from the previous activity (ILO B.2) but added this:

```cpp
template<typename T>
bool match(T open, T close){
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

template<typename T>
bool balanced(std::string expression){

    while(!isEmpty<T>())
        pop<T>();

    for(char ch : expression){

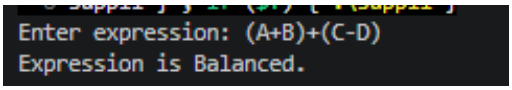
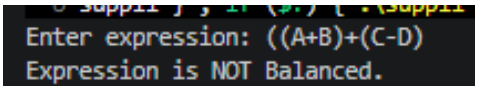
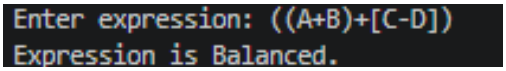
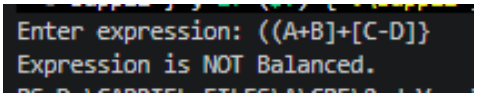
        if(ch == '(' || ch == '{' || ch == '['){
            push<T>(ch);
        }
        else if(ch == ')' || ch == '}' || ch == ']'){

            if(isEmpty<T>())
                return false;

            char temp = pop<T>();

            if(!match<char>(temp, ch))
                return false;
        }
    }

    return isEmpty<T>();
}
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		The linked list stack stores each opening symbol as a new node. When a closing symbol is encountered, the top node is removed and checked. Since all symbols match properly, the stack becomes empty and the expression is valid.
((A+B)+(C-D)	N		The program finds that there is still an opening parenthesis stored in the linked list after scanning the entire expression. This means there is a missing closing parenthesis, making the expression invalid.
((A+B)+[C-D])	Y		The linked list implementation correctly manages the pushed and popped symbols. The symbols are removed in the correct order, and all pairs are matched successfully.
((A+B]+[C-D]}	N		The expression contains mismatched symbols. The popped opening symbols do not correspond with the closing symbols being checked, so the program identifies the expression as not balanced.

Tools Analysis:

● How do the different internal representations affect the implementation and usage of the stack?

In the array implementation, the stack is easier to understand because the elements are stored in a fixed-size array and the topIndex is used to track the current top element. However, the size is limited because the array has a fixed capacity. For the linked list implementation, the stack is more flexible because it can grow and shrink depending on the number of elements added. Instead of using an index, it uses nodes and pointers to connect each element. Although it uses more memory because of the extra pointer, it avoids the size limitation of arrays. Both implementations follow the same stack operations such as push and pop, but the way they store and manage data is different.

Summary of Lessons Learned, Analysis, and Conclusion

In this activity, I learned more about how stacks work and how they can be implemented using different data structures such as arrays and linked lists. For the array implementation, I understood how the topIndex is used to

track the top element and how push and pop operations affect the stack. For the linked list implementation, I learned how nodes and pointers are used to dynamically add and remove elements without having a fixed size.

The supplementary activity helped me understand the real application of stacks through checking balanced symbols. I learned that the Last In, First Out (LIFO) concept is useful when dealing with nested symbols like parentheses, brackets, and braces because the most recent opening symbol should be the first one checked. The hardest part for me was creating the bool balanced() function because I initially did not fully understand the logic behind the algorithm. I used AI as a guide to help me understand how the function works, especially how pushing opening symbols and popping them when a closing symbol appears allows the program to check if the expression is valid. After reviewing the process, I was able to understand the purpose of each step and apply it to my code.

I think I did fairly well in this activity because I was able to modify my previous stack implementations and apply them to a new problem instead of starting from scratch. However, I still need to improve my understanding of algorithms before coding, especially when working with more complex logic. I also want to practice more with pointers and linked lists since managing nodes and memory can sometimes be confusing. Overall, this activity improved my understanding of stacks and showed me how they are used in actual programming problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI as a guide to help me understand the concept of the bool balanced() function and the logic behind checking symbols in a stack. It helped explain difficult parts of the code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.